Lab 3 — Task 1: Configure Infrastructure and Automate Application Using Ansible

Akash Singh Sagar | Repository: github.com/akashsinghsagar/devops-lab3

Objective

Configure infrastructure and automate application configuration using Ansible: install Ansible on a control node, define an inventory of target nodes, write a playbook, and apply it to the target nodes.

Step 1 — Install Ansible on the control node

Ansible is installed from the Ubuntu repositories on the control node and the installation is verified.

```
sudo apt update
sudo apt install -y ansible
ansible --version
```

```
akash@PHANTOM:/mnt/c/Users/ARSH$ sudo apt update
sudo apt install -y ansible
ansible --version
Ign:1 https://pkg.jenkins.io/debian-stable binary/ InRelease
Hit:2 https://pkg.jenkins.io/debian-stable binary/ Release
Hit:4 http://security.ubuntu.com/ubuntu resolute-security InRelease
Hit:5 http://archive.ubuntu.com/ubuntu resolute InRelease
Hit:6 http://archive.ubuntu.com/ubuntu resolute-updates InRelease
Hit:7 http://archive.ubuntu.com/ubuntu resolute-backports InRelease
97 packages can be upgraded. Run 'apt list --upgradable' to see them.
Installing:
  ansible

Installing dependencies:
  ansible-core          python3-dnspython  python3-libcloud       python3-resolvelib
  ieee-data             python3-gssapi     python3-netaddr        python3-selinux
  python3-argcomplete   python3-jmespath   python3-pyspnego       python3-winrm
  python3-decorator     python3-kerberos   python3-requests-ntlm  python3-xmltodict

Suggested packages:
  cowsay   python3-aioquic  python3-httpcore  python3-trio  python-netaddr-docs
  sshpass  python3-h2       python3-httpx     ipython3

Summary:
  Upgrading: 0, Installing: 17, Removing: 0, Not Upgrading: 97
  Download size: 25.1 MB
  Space needed: 262 MB / 1023 GB available
```

Figure 1 — Ansible installed and version verified on the control node.

Step 2 — Create the inventory

The inventory lists the target node under a webservers group, with the SSH user and private key used to connect.

```
[webservers]
node1 ansible_host=<TARGET_PRIVATE_IP> ansible_user=ubuntu
ansible_ssh_private_key_file=~/.ssh/<your-key>.pem
```

Step 3 — Test connectivity

The ping module confirms that Ansible can reach and authenticate to the target node.

```
ansible all -i inventory.ini -m ping
```

```
akash@PHANTOM:~/devops-lab3$ ansible all -i inventory.ini -m ping
[WARNING]: Host 'localhost' is using the discovered Python interpreter at '/usr/bin/python3.14', but f
uture installation of another Python interpreter could cause a different interpreter to be discovered.
 See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html for mo
re information.
localhost | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3.14"
    },
    "changed": false,
    "ping": "pong"
}
```

Figure 2 — Target node responds with "pong", confirming SSH connectivity.

Step 4 — Write the playbook

The playbook (site.yml) updates the package cache, installs nginx and git, deploys an index page, and ensures nginx is enabled and running. A handler restarts nginx only when the page changes.

```
akash@PHANTOM:/mnt/c/Users/ARSH$ mkdir -p ~/devops-lab3 && cd ~/devops-lab3

cat > inventory.ini <<'EOF'
[webservers]
localhost ansible_connection=local
EOF

cat > site.yml <<'EOF'
---
- name: Configure web server and deploy application
  hosts: webservers
  become: true
  vars:
    app_message: "Deployed with Ansible by Akash Singh Sagar"

  tasks:
    - name: Update apt cache
      ansible.builtin.apt:
        update_cache: true
        cache_valid_time: 3600

    - name: Install nginx and git
      ansible.builtin.apt:
        name: [nginx, git]
        state: present

    - name: Deploy index page
      ansible.builtin.copy:
        content: "<h1>{{ app_message }}</h1>\n"
        dest: /var/www/html/index.html
        mode: "0644"
      notify: Restart nginx

    - name: Ensure nginx is enabled and running
      ansible.builtin.service:
        name: nginx
        state: started
        enabled: true

  handlers:
    - name: Restart nginx
      ansible.builtin.service:
        name: nginx
        state: restarted
EOF
akash@PHANTOM:~/devops-lab3$
```

Step 5 — Apply the playbook

The syntax is checked first, then the playbook is applied to the target node.

```
$ ansible-playbook -i inventory.ini site.yml --syntax-check
ansible-playbook -i inventory.ini site.yml
```

```
akash@PHANTOM:~/devops-lab3$ ansible-playbook -i inventory.ini site.yml

PLAY [Configure web server and deploy application] ***************************************************

TASK [Gathering Facts] *******************************************************************************
[WARNING]: Host 'localhost' is using the discovered Python interpreter at '/usr/bin/python3.14', but f
uture installation of another Python interpreter could cause a different interpreter to be discovered.
 See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html for mo
re information.
ok: [localhost]

TASK [Update apt cache] ******************************************************************************
ok: [localhost]

TASK [Install nginx and git] *************************************************************************
changed: [localhost]

TASK [Deploy index page] *****************************************************************************
changed: [localhost]

TASK [Ensure nginx is enabled and running] ***********************************************************
ok: [localhost]

RUNNING HANDLER [Restart nginx] **********************************************************************
changed: [localhost]

PLAY RECAP *******************************************************************************************
localhost                  : ok=6    changed=3    unreachable=0    failed=0    skipped=0    rescued=0
   ignored=0
```

Figure 3 — Playbook applied to the target node; PLAY RECAP shows the tasks that changed.

Step 6 — Verify idempotency

Running the same playbook a second time should report no changes, showing the node is already in the desired state.

```
ansible-playbook -i inventory.ini site.yml
```

Figure 3 — Second run reports changed=0, confirming idempotency.

Result

Ansible was installed on the control node, an inventory and playbook were written, and the playbook was applied to the target node to install and configure nginx and deploy the application page. A repeat run confirmed the configuration is idempotent. Task 1 is complete.